Toxic Comment Classification

Seminar project, topic 3: multi-label classification of the Jigsaw corpus

Group members: *(names and student IDs)*

Course teacher: Assoc. Prof. Phuong T. Nguyen, University of L'Aquila

October 26, 2026

Abstract

We study the Jigsaw Toxic Comment Classification Challenge, a multi-label task in which each of 159,571 Wikipedia talk-page comments may carry any subset of six labels: *toxic*, *severe_toxic*, *obscene*, *threat*, *insult* and *identity_hate*. We compare two trivial baselines, three linear classifiers over word and character TF-IDF features and a fine-tuned DistilBERT under 5-fold multi-label stratified cross validation, with per-label decision thresholds tuned inside each training fold. The best model, distilbert, reaches a mean column-wise ROC-AUC of 0.9898 ± 0.0011, ahead of the best linear model, logreg (0.9841), and far ahead of a profanity-list baseline (0.78). The aggregate hides two weaknesses: the rarest labels remain hard (PR-AUC between 0.50 and 0.61 for *severe_toxic*, *threat* and *identity_hate*), and both model families are biased against identity terms – a clean comment mentioning *gay* is flagged as toxic 6.8 times as often as the average clean comment by DistilBERT, and 10.5 times by logistic regression.

Contents

1 Introduction **3**

2 Dataset **3**
- 2.1 Label frequency . . . 3
- 2.2 Label co-occurrence and hierarchy . . . 3
- 2.3 Comment length . . . 4

3 Methodology **4**
- 3.1 Preprocessing . . . 4
- 3.2 Models . . . 5
- 3.3 Cross validation . . . 5
- 3.4 Threshold tuning . . . 5
- 3.5 Metrics . . . 5
- 3.6 Statistical comparison . . . 6

4 Implementation **6**
- 4.1 Persistence layer . . . 6

5 Results **7**
- 5.1 Model comparison . . . 7
- 5.2 Per-label results . . . 8
- 5.3 Effect of threshold tuning . . . 9
- 5.4 Official test set . . . 10
- 5.5 Statistical significance . . . 10
- 5.6 Cost against accuracy . . . 11

6 Error analysis **11**
6.1 Confusion matrices 11
6.2 Error taxonomy 12
6.3 Label hierarchy violations 13

7 Unintended bias **13**

8 Limitations **15**

9 Conclusion **15**

A Reproducibility **15**

B Division of work **16**

1 Introduction

Online platforms rely on automatic moderation to keep discussions civil, and the first step of any such system is a classifier that recognises abusive language. The Jigsaw Toxic Comment Classification Challenge, hosted on Kaggle by Jigsaw and Google in 2018, frames this as a *multi-label* problem: a comment is not merely toxic or clean, it can be at once obscene, insulting and hateful towards an identity group.

The project brief asks us to implement machine learning or deep learning classifiers, evaluate them with suitable metrics under k-fold cross validation, compare them with dedicated baselines, and analyse the results. Our contributions are:

- a reproducible pipeline in Python that cross-validates every model on one shared fold assignment, so that models can be compared pairwise;
- a comparison of six models, from a constant predictor to a fine-tuned transformer, with the competition's metric and with metrics that expose the rare labels;
- an analysis of what the best model gets wrong: an error taxonomy built from concrete examples, violations of the label hierarchy, and an audit of unintended bias against identity terms.

2 Dataset

The training set contains 159,571 comments, each with six binary labels. The competition's test set is labelled only partially and only after the competition closed (63,978 of 153,164 rows are usable), so, as the brief requires, we evaluate exclusively by cross validation on the training set.

2.1 Label frequency

The labels are rare and severely imbalanced (Figure 1): 89.8% of the comments carry no label at all, *toxic* occurs in 9.6% of the comments, and *threat* in only 0.3% – 478 comments in the whole corpus, about 96 per validation fold. Accuracy is therefore meaningless: the constant "clean" prediction is right on nine comments out of ten.

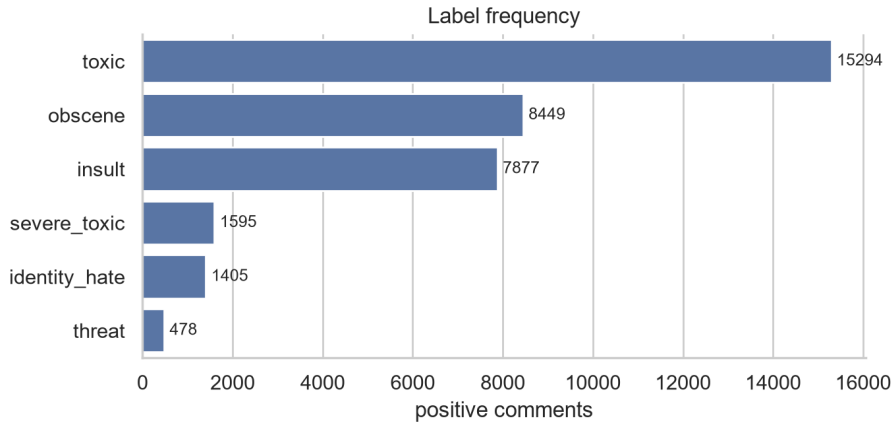


Figure 1: Number of positive comments per label.

2.2 Label co-occurrence and hierarchy

The labels are strongly correlated (Figure 2). *severe_ toxic* is a strict subset of *toxic*: $P(\text{toxic} \mid \text{severe_toxic}) = 1$. The other four labels co-occur with *toxic* in 93–94% of cases but not always

(for instance 523 obscene comments are not labelled toxic). A binary-relevance model, which trains each label independently, ignores this structure; Section 6.3 measures what that costs.

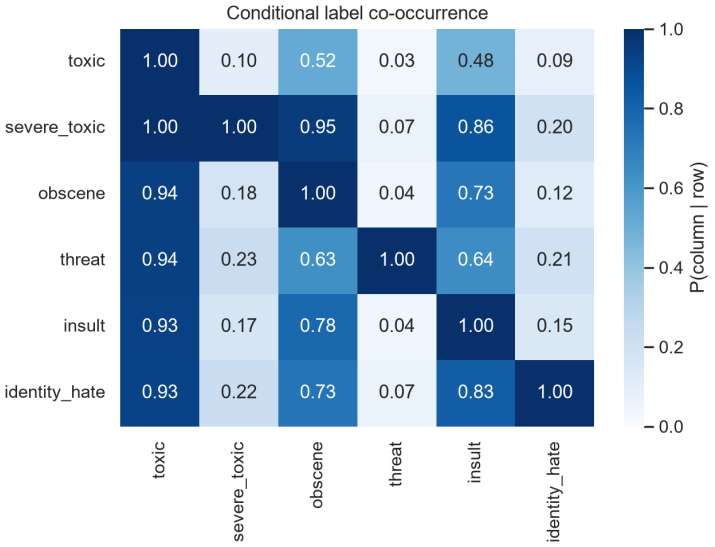


Figure 2: Conditional co-occurrence $P(\text{column} \mid \text{row})$.

2.3 Comment length

Comments are long-tailed (Figure 3): the median comment has 36 words, the 95th percentile 230 and the longest 1,411. This informs the transformer's maximum sequence length of 192 sub-word tokens, which keeps most comments whole.

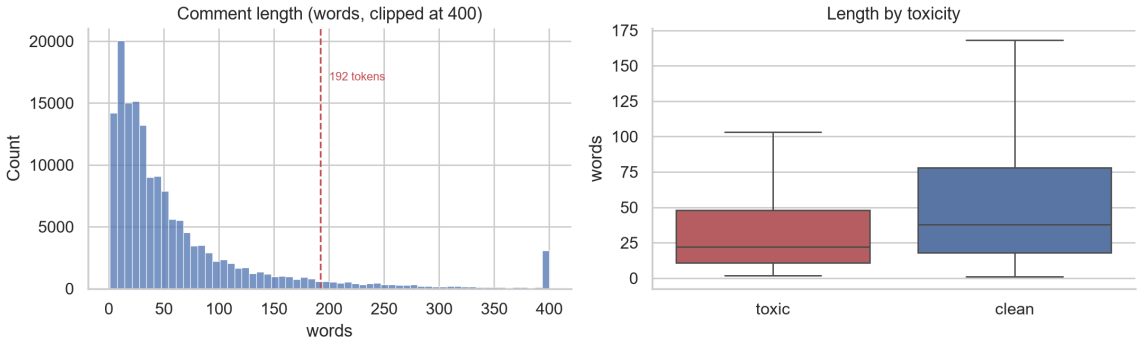


Figure 3: Distribution of comment length, and length by toxicity.

3 Methodology

3.1 Preprocessing

The comments are raw wiki markup written by anonymous editors. A configurable cleaner, implemented as a scikit-learn transformer so that it runs inside each fold, applies in order: removal of user signatures and escaped newlines; replacement of URLs and IP addresses with placeholder tokens; lower-casing; expansion of English contractions; reversal of common obfuscations (`sh!t`, `b1tch`), where digits are rewritten only inside tokens that also contain letters so that numbers survive; removal of punctuation; and collapsing of character runs longer than two (`stuuuupid` → `stuupid`), which keeps the emphasis signal. The transformer receives the raw text, since its sub-word tokenizer handles casing and punctuation itself.

3.2 Models

All models output a score in $[0, 1]$ for each of the six labels.

Baselines. *prior* predicts each label's training frequency for every comment; its ROC-AUC is 0.5 by construction and it shows what accuracy-like metrics reward on imbalanced data. *keyword* flags a comment when it contains a word from a fixed profanity list: it is the rule-based system a team would build without machine learning, and therefore the honest reference point for whether learning is worth it.

Linear models. The three linear models share one feature space: the union of a word TF-IDF (unigrams and bigrams, 50,000 features) and a character TF-IDF (2- to 5-grams within word boundaries, 50,000 features), both with sublinear term frequency. Character n-grams are robust to the misspellings and obfuscations that dominate abusive text. Following the binary-relevance approach, one binary classifier is fitted per label, with balanced class weights:

- *logreg*: logistic regression;
- *linear_svm*: a linear support vector machine, whose decision function is squashed through a sigmoid to obtain a score;
- *nbsvm*: the NB-SVM of Wang and Manning [1], which multiplies the features by the Naive Bayes log-count ratio $r = \log\big(P(f \mid y{=}1)/P(f \mid y{=}0)\big)$ before fitting a logistic regression. It was among the strongest single models of the competition.

Transformer. *distilbert* fine-tunes DistilBERT [2] (`distilbert-base-uncased`) with a six-unit sigmoid head under binary cross-entropy, for two epochs, with AdamW (learning rate $3 \cdot 10^{-5}$, weight decay 0.01), batch size 32, sequences truncated to 192 tokens, gradient clipping at 1.0 and mixed precision on the GPU. Unlike the bag-of-n-grams models it sees word order and context, which is what can separate "this article is garbage" from "you are garbage".

3.3 Cross validation

Plain stratified k-fold is defined for a single target. We use *iterative stratification* [3], which assigns comments to folds so that the frequency of every label, including the rare ones, is balanced across folds. We use $k = 5$, and all models are evaluated on the *same* fold assignment, which makes a paired comparison of their per-fold scores valid.

3.4 Threshold tuning

Turning scores into labels needs a threshold per label. Instead of the default 0.5, we choose the threshold that maximises F1 on an inner hold-out of 15% of each *training* fold; the model is fitted on the remaining 85%. The validation fold is never used to choose a threshold, so the reported F1 is not optimistically biased.

3.5 Metrics

- **Mean column-wise ROC-AUC**, the competition's official metric: the ROC-AUC of each label, averaged. It is threshold-free and insensitive to class imbalance.
- **PR-AUC** (average precision) per label and averaged. On a label with a 0.3% positive rate, ROC-AUC is dominated by the easy negatives; PR-AUC measures how precise the model is on the positives, and is the more discriminating metric for rare labels.

- **Macro F1** (unweighted mean over labels, so rare labels count as much as frequent ones) and **micro F1** (pooled over all label decisions), at the tuned thresholds.
- **Hamming loss** and **subset accuracy**, the standard multi-label metrics, computed for every fold and stored in `results/` for completeness.

3.6 Statistical comparison

For each pair of models we apply the Wilcoxon signed-rank test to their per-fold mean column-wise AUCs, with a Bonferroni correction for the number of pairs. With five folds the smallest attainable two-sided p-value is $2/2^5 = 0.0625$, reached when one model wins on every fold; no pairwise difference can therefore be significant at the 5% level, and we report the consistency of the ranking across folds alongside the p-values.

4 Implementation

The application is written in Python and organised as a package (`src/`) used by command-line scripts and by three Jupyter notebooks, which also run on Google Colab.

- `dataio`: loading and schema validation of the Kaggle files, and a synthetic generator used to smoke-test the pipeline without the data.
- `preprocessing`: the text cleaner and the TF-IDF vectorisers.
- `models`: a common `fit`/`predict_proba` interface implemented by every model, and a registry that builds a model from its name.
- `evaluation`: multi-label folds, cross validation, metrics, threshold optimisation, statistical tests, the hierarchy check and the bias audit.
- `repository`: persistence of datasets and results behind one interface with two implementations, file system and in memory (Section 4.1).

4.1 Persistence layer

The course requires that data can be managed both in memory and through files, and that switching between the two leaves the classes, their methods and the business logic unchanged. We meet this with the *repository* pattern. Two abstract base classes define what can be done with stored data, without saying where it is stored:

```
class DatasetRepository(ABC):          # tabular data (pandas.DataFrame)
    def save(self, name: str, frame: pd.DataFrame) -> None: ...
    def load(self, name: str) -> pd.DataFrame: ...
    def exists(self, name: str) -> bool: ...
    def list_datasets(self) -> list[str]: ...
    def delete(self, name: str) -> None: ...

class ArtifactRepository(ABC):         # results: metrics, scores, models
    def save(self, name: str, obj: Any) -> None: ...
    def load(self, name: str) -> Any: ...
    def exists(self, name: str) -> bool: ...
    def list_artifacts(self) -> list[str]: ...
    def delete(self, name: str) -> None: ...
```

Each has two implementations. The file-system ones write CSV or Parquet files for datasets, and JSON (or `joblib` for objects JSON cannot represent) for artifacts; the in-memory ones keep the objects in a dictionary and return deep copies of data frames and of plain containers, so that a caller cannot mutate stored data by accident. A factory is the only code that names a concrete class:

```
def get_dataset_repository(config) -> DatasetRepository:
    if config.persistence_backend == "memory":
        return _MEMORY_DATASETS              # process-wide singleton
    return FileDatasetRepository(root=config.processed_dir)
```

Every other module – the data loader, the experiment runner, the report scripts, the notebooks – obtains its repository from the factory and is typed against the abstract interface only. Switching the whole application between the two layers is therefore a single value: the `persistence_backend` field of the configuration, the `TCC_PERSISTENCE` environment variable, or the `-backend file|memory` flag of the experiment scripts. The repository tests are parametrised to run unchanged against both implementations, and the suite includes a test that executes one piece of business logic under each backend and checks that the results are identical. In practice we use the memory backend for quick notebook sessions and for the tests, and the file backend for the experiments, whose results must outlive the process.

`scripts/run_experiment.py` runs the cross validation and stores every artifact (per-fold metrics, out-of-fold scores, thresholds, significance matrix, bias audit) in `results/`. Because the transformer needs a GPU, it is run separately, and `scripts/merge_runs.py` folds its results into the main run after checking that both used the same folds. The tables and figures of this report are generated from those files by two scripts, never copied by hand. A test suite of 80 `pytest` tests covers the data layer, the preprocessing, the models, the metrics, the cross validation and both persistence backends.

5 Results

5.1 Model comparison

Table 1 and Figure 4 rank the models by the official metric.

Table 1: Cross-validated results, mean over 5 folds (± standard deviation for AUC). F1 at the tuned thresholds. Fit time per fold.

Model	Mean col. AUC	PR-AUC	Macro F_1	Micro F_1	Fit (s)
distilbert	0.9898 ± 0.0011	0.7193	0.6827	0.7870	2238.4
logreg	0.9841 ± 0.0010	0.6726	0.6410	0.7551	232.0
nbsvm	0.9830 ± 0.0010	0.6665	0.6350	0.7566	238.0
linear_svm	0.9800 ± 0.0008	0.6559	0.6306	0.7486	203.8
keyword	0.7831 ± 0.0075	0.1298	0.2485	0.2821	0.6
prior	0.5000 ± 0.0000	0.0367	0.0616	0.1231	0.0

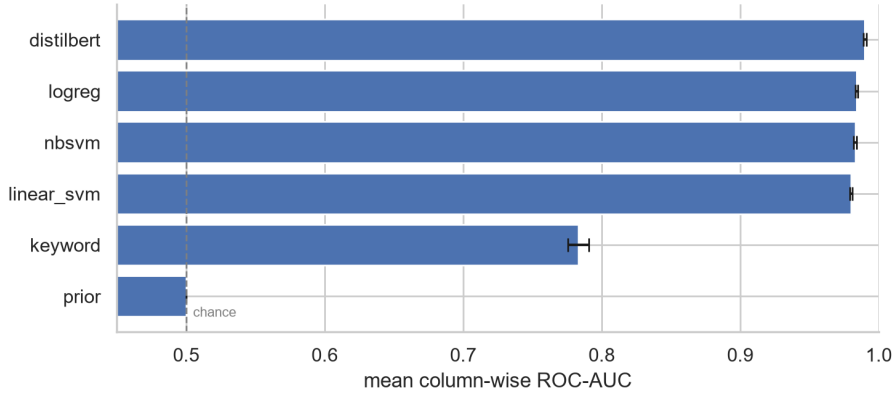


Figure 4: Mean column-wise ROC-AUC per model, with the standard deviation over folds.

Four observations stand out.

Learning is worth it. The keyword baseline reaches an AUC of 0.78 but a macro F1 of only 0.25; every learned model exceeds 0.98 AUC and 0.63 macro F1. The prior baseline sits at exactly 0.5 AUC, as expected.

The transformer is the best model on every metric. DistilBERT reaches 0.9898 AUC against 0.9841 for the best linear model, and it wins on each of the five folds (fold AUCs 0.9889–0.9916 against 0.9827–0.9850). The gain is larger where it matters most: macro F1 rises from 0.641 to 0.683 and mean PR-AUC from 0.673 to 0.719, mainly through the rare labels (Section 5.2).

Among the linear models the differences are small but consistent. logreg leads with 0.9841, ahead of nbsvm (0.9830) and linear_svm (0.9800). The gap between logreg and nbsvm is only 0.001 AUC, yet logreg wins on all five folds. In our feature space, which already includes character n-grams, the NB rescaling brings no gain.

All learned models are close to the competition's best. The winning private-leaderboard score was about 0.988, obtained with large ensembles of deep models. In cross validation a single logistic regression comes within 0.005 of it and a single DistilBERT matches it, but cross validation is optimistic: on the official test set the logistic regression scores 0.9784 (Section 5.4).

5.2 Per-label results

The aggregate AUC hides how unequal the labels are (Table 2, Figure 5). For DistilBERT, ROC-AUC is above 0.98 on every label, but PR-AUC ranges from 0.92 on *obscene* and *toxic* to 0.50 on *severe_toxic*, 0.54 on *threat* and 0.61 on *identity_hate*. The frequent, lexically marked labels are easy; the rare ones are hard both because they have few training examples and because they depend on meaning rather than vocabulary – a threat need not contain a single offensive word, and *severe_toxic* differs from *toxic* by degree. The transformer's largest gains over logreg are on two of these rare labels: +0.11 PR-AUC on *identity_hate* and +0.05 on *severe_toxic*, against +0.015 to +0.036 on the other four.

Table 2: Per-label results of distilbert, mean over folds. Support is the number of positives per validation fold.

Label	Support	ROC-AUC	PR-AUC	Prec.	Recall	F_1	Thresh.
toxic	3,059	0.9861	0.9160	0.8357	0.8283	0.8317	0.494
severe_toxic	319	0.9914	0.4998	0.4408	0.6395	0.5193	0.265
obscene	1,690	0.9936	0.9185	0.8378	0.8416	0.8391	0.482
threat	96	0.9901	0.5374	0.5413	0.5919	0.5637	0.255
insult	1,575	0.9887	0.8296	0.7281	0.8060	0.7648	0.414
identity_hate	281	0.9890	0.6145	0.5591	0.6028	0.5778	0.380

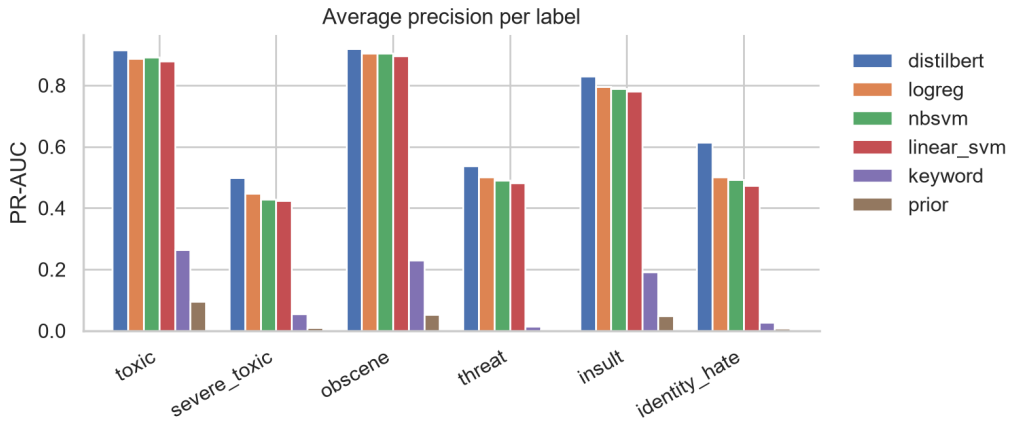


Figure 5: PR-AUC per label and model.

5.3 Effect of threshold tuning

Table 3 compares F1 at the default cut-off of 0.5 with F1 at the tuned threshold. For DistilBERT the tuned thresholds lie *below* 0.5, between 0.25 and 0.49, and they matter a great deal on the rare labels: +0.17 F1 on *severe_toxic* and +0.11 on *threat*. Trained with an unweighted loss, the transformer is cautious about rare labels and its scores for them tend to stay below 0.5; the tuned threshold corrects this.

The linear models behave in the opposite way. They are trained with balanced class weights, which inflate the scores of the positive class: logreg's tuned thresholds lie between 0.60 and 0.75, and tuning gains only −0.006 to +0.023 F1, because the class weights have already done most of the correction. Threshold tuning and class weighting are thus two answers to the same imbalance, and one of them is necessary.

Table 3: F1 of distilbert at 0.5 and at the tuned threshold.

Label	Tuned thresh.	F_1 @ 0.5	F_1 @ tuned	Gain
toxic	0.494	0.8291	0.8317	+0.0026
severe_toxic	0.265	0.3459	0.5193	+0.1734
obscene	0.482	0.8379	0.8391	+0.0013
threat	0.255	0.4548	0.5637	+0.1089
insult	0.414	0.7491	0.7648	+0.0157
identity_hate	0.380	0.5666	0.5778	+0.0113

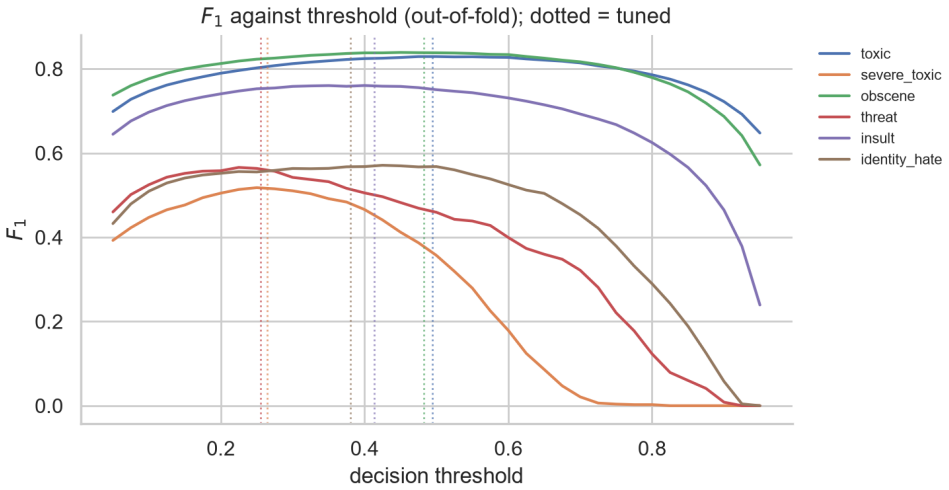


Figure 6: F1 as a function of the decision threshold, out-of-fold scores.

5.4 Official test set

Cross validation compares models with each other; it does not say how a model would have fared in the competition. For that we scored the models once on the 63,978 test comments that Kaggle labelled after the competition closed (`scripts/evaluate_test_set.py`). Each model is trained as in one fold: fitted on 85% of the full training set, with its thresholds tuned on the remaining 15%. The transformer was not re-trained for this experiment, since that needs another GPU run.

Table 4: Results on the scored part of the official test set.

Model	Mean col. AUC	PR-AUC	Macro F_1	Micro F_1
logreg	0.9784	0.6098	0.5763	0.6486
nbsvm	0.9781	0.6003	0.5573	0.6370
linear_svm	0.9746	0.5933	0.5507	0.6348
keyword	0.7815	0.1131	0.2177	0.2450
prior	0.5000	0.0378	0.0641	0.1280

The ranking of the models is the same as in cross validation, but every learned model scores lower: logreg reaches 0.9784 AUC on the test set against 0.9841 in cross validation, and the other linear models lose about 0.005 as well. Macro F1 drops more (0.641 to 0.576 for logreg), because thresholds tuned on training data transfer imperfectly to the test comments. Cross validation on the training set is thus optimistic for this task, and the gap between our best linear model and the winning private-leaderboard score (about 0.988) is 0.010 rather than the 0.005 that the cross-validated number suggests.

5.5 Statistical significance

Table 5 gives the pairwise Wilcoxon p-values. Every pair reaches the minimum attainable value of 0.0625: for each pair of models, one of the two is better on all five folds. The ranking is thus perfectly consistent across folds, but with five folds this cannot be declared significant at the 5% level, let alone at the Bonferroni-corrected level of $0.05/15 = 0.0033$. Establishing significance would require more folds or repeated cross validation.

Table 5: Pairwise Wilcoxon signed-rank p-values on the per-fold mean column-wise AUC. Bold: significant at the Bonferroni-corrected level.

Model	prior	keyword	logreg	nbsvm	linear_svm	distilbert
prior	—	0.062	0.062	0.062	0.062	0.062
keyword	0.062	—	0.062	0.062	0.062	0.062
logreg	0.062	0.062	—	0.062	0.062	0.062
nbsvm	0.062	0.062	0.062	—	0.062	0.062
linear_svm	0.062	0.062	0.062	0.062	—	0.062
distilbert	0.062	0.062	0.062	0.062	0.062	—

5.6 Cost against accuracy

Figure 7 relates accuracy to training cost. The three linear models train in 3.5–4 minutes per fold on a laptop CPU, about 400 times the cost of the keyword rule but still cheap enough to retrain daily. DistilBERT needs 37 minutes per fold on average on a laptop GPU (an NVIDIA RTX 4050 with mixed precision), about ten times as long; individual folds took between 19 and 82 minutes. The 0.006 AUC it gains costs an order of magnitude more compute and a GPU; whether that is worth it depends on how much the rare labels matter to the application.

The two families were timed in separate runs on different machines – the linear models on a laptop CPU, the transformer on a laptop GPU – so the ratio is indicative only.

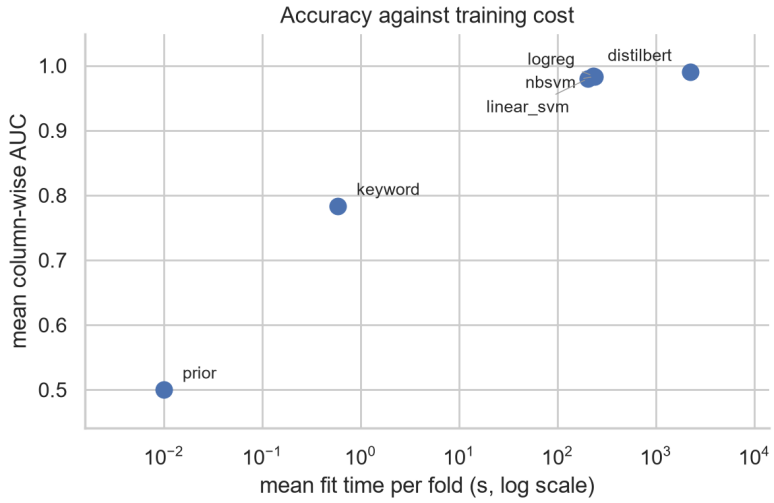


Figure 7: Mean column-wise AUC against fit time per fold (log scale).

6 Error analysis

6.1 Confusion matrices

Figure 8 shows the confusion matrix of each label at its tuned threshold, pooled over the out-of-fold predictions of DistilBERT. On *toxic* it makes 2,500 false positives and 2,626 false negatives out of 15,294 toxic comments, against 2,766 and 3,301 for logreg.

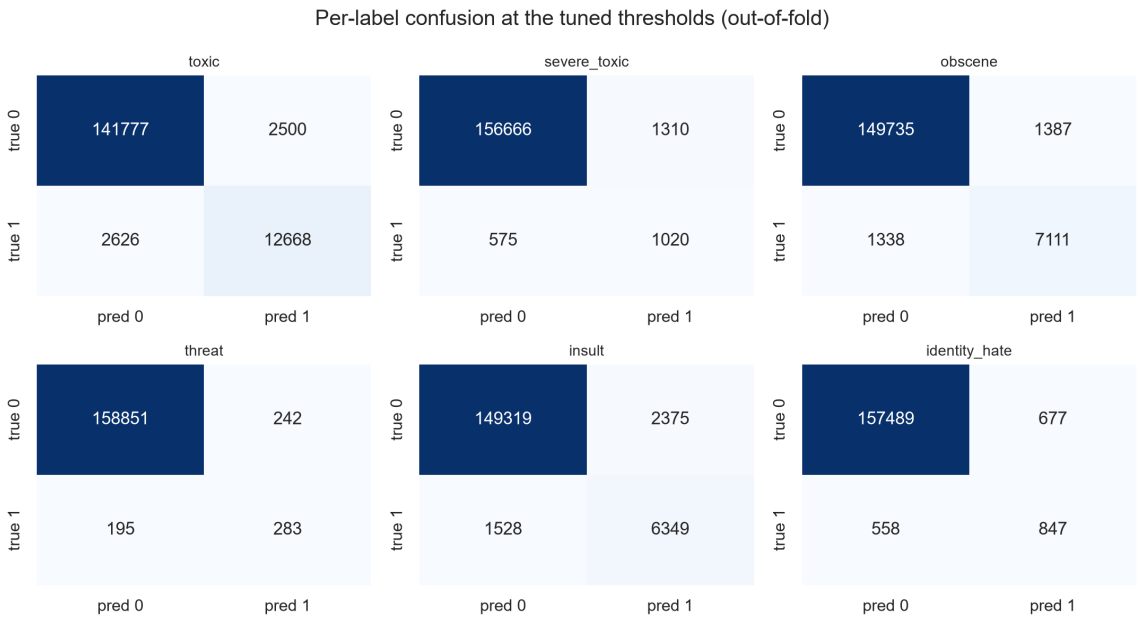


Figure 8: Confusion matrices at the tuned thresholds, out-of-fold.

6.2 Error taxonomy

We read the false positives and false negatives on *toxic* that each of logreg and DistilBERT was most confident about (`scripts/make_error_examples.py`), and compared the two models' scores on the same comments. The errors fall into a few recurring categories.

False positives (clean comments flagged as toxic).

- *Annotation noise.* Most of the confident "errors" of both models are plainly abusive comments – direct personal insults, often with slurs ("you're an asshole and that's all there is to it") – that the annotators left unlabelled. Here the model is right and the label is wrong. For DistilBERT, ten of its twelve most confident false positives are of this kind.
- *Profanity without abuse.* Swear words used descriptively or about oneself. A list of TV content ratings quoting the words they warn about scores 1.00 under logreg but 0.45 under DistilBERT, which is below its threshold: reading the context, the transformer recognises the quotation. A comment about oneself ("I'm going through deep shit") still fools both (1.00 and 0.83).
- *Mild incivility.* Rude but borderline comments ("It must suck to be you then") on which annotators disagree.

False negatives (toxic comments missed).

- *Annotation noise.* Comments with no hostility at all – a list of a band's musical influences, a note about magazine-cover images – are labelled toxic. Both models score them near zero, rightly.
- *Shouting and heated disagreement.* Comments written in capitals or accusing other editors, with no offensive vocabulary ("STOP DELETING MY ADDITION!"). Both models miss them (0.00 and 0.04).
- *Sarcasm.* "Thanks for participating in the conspiracy against me." The words are polite and the intent is not; both models score it 0.00.

- *Harassment without profanity, and toxicity diluted by length.* A sexual remark about a named person in innocuous words, or a short insult followed by a long pasted article. DistilBERT raises these from about 0.00 under logreg to 0.17–0.20, but not above its threshold.

The transformer's advantage is visible in exactly the categories that depend on context: who a swear word is aimed at, whether a sentence is a quotation. Sarcasm and shouting remain beyond both models. And a large share of the remaining confident errors are labelling mistakes, which bound the score that any model can reach on this data.

6.3 Label hierarchy violations

The linear models predict each label independently, so nothing prevents them from predicting a sub-type without *toxic*. logreg does so for 4 of its 2,503 *severe_ toxic* predictions – outputs that are impossible, since the label never occurs without *toxic* – and for about 7% of its *threat* and *identity_ hate* predictions, close to the 6–7% at which these labels occur without *toxic* in the data.

DistilBERT shares one encoder across the six outputs, and it behaves differently (Table 6): it never predicts *severe_ toxic* or *insult* without *toxic*, and does so for at most 0.4% of the other sub-labels. It has thus learnt the hierarchy – and in fact over-learnt it, since in the data 6–7% of the *obscene*, *threat*, *insult* and *identity_ hate* comments are *not* labelled toxic. For the linear models, a post-processing rule that sets *toxic* whenever *severe_ toxic* is predicted would remove the impossible outputs.

Table 6: Predicted positives of each sub-label that lack a *toxic* prediction, distilbert, out-of-fold.

Label	Predicted positive	Without *toxic*	Rate
severe_toxic	2,330	0	0.00%
obscene	8,498	13	0.15%
threat	525	2	0.38%
insult	8,724	0	0.00%
identity_hate	1,524	4	0.26%

7 Unintended bias

A model trained on this corpus sees identity terms mostly in abusive contexts and can learn to penalise the term itself. We measure this as follows: for each identity term, we take the clean comments that mention it and compute the false-positive rate of *toxic* at the tuned threshold; dividing by the false-positive rate on all clean comments gives a ratio that is 1 for an unbiased model.

Table 7: False-positive rate on *toxic* for clean comments mentioning each identity term, against the background rate (distilbert, out-of-fold).

Term	n	Toxic rate	Subgroup FPR	Background FPR	Ratio	Subgroup AUC
gay	1,002	0.547	0.117	0.017	6.77	0.953
lesbian	79	0.304	0.109	0.018	6.23	0.939
transgender	31	0.194	0.080	0.018	4.56	0.973
black	1,458	0.123	0.043	0.017	2.48	0.953
muslim	744	0.105	0.030	0.017	1.72	0.946
white	1,512	0.120	0.029	0.017	1.68	0.965
woman	543	0.125	0.025	0.018	1.44	0.956
jewish	763	0.092	0.025	0.018	1.40	0.955
christian	1,175	0.067	0.020	0.018	1.14	0.963
female	396	0.096	0.020	0.018	1.11	0.974
immigrant	104	0.106	0.000	0.018	0.00	0.992

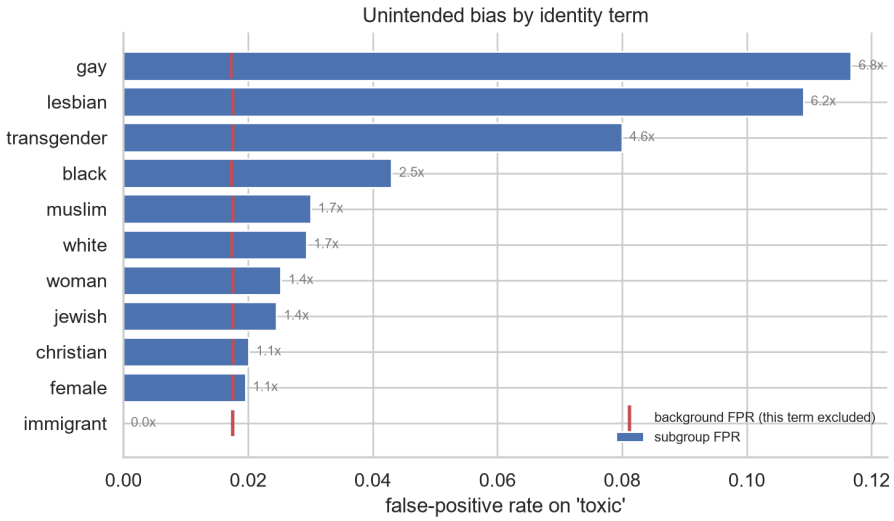


Figure 9: False-positive rate on *toxic* for clean comments mentioning each term (bars), against the background rate (red marks), with the ratio of the two; distilbert, out-of-fold.

The bias is large (Table 7, Figure 9). For DistilBERT, a clean comment mentioning *gay* is flagged as toxic 6.8 times as often as the average clean comment, *lesbian* 6.2 times, *transgender* 4.6 times (on only 31 comments) and *black* 2.5 times. The cause is visible in the data: 55% of the comments mentioning *gay* are toxic, against 10% overall, so the term itself is a strong predictor. The subgroup AUC stays high (0.94–0.99), which means the model still ranks toxic above non-toxic comments *within* each group; the problem is that the whole group is shifted upwards.

The bag-of-n-grams model is markedly worse. Under logreg the ratio is 10.5 for *gay* and 6.7 for *lesbian*, and *woman*, *white*, *female* and *transgender* are flagged about twice as often as the background. Reading context halves several of these ratios – *woman* falls from 2.7 to 1.4, *female* from 2.1 to 1.1 – but does not remove the bias for the most targeted groups.

In a moderation system this means that people talking about their own identity are silenced more often. Remedies include augmenting the training data with neutral sentences containing the terms, reweighting the subgroups, or a fairness-aware objective; this is the problem that motivated Jigsaw's follow-up competition on unintended bias.

8 Limitations

- *Five folds cannot establish significance* (Table 5); repeated cross validation would.
- *The cleaner's de-obfuscation is aggressive.* It rewrites every "!" as "i" to undo `sh!t`, which also turns an ordinary "world!" into "worldi". The character n-grams make the models robust to this, but the rule should be restricted to tokens that contain letters on both sides.
- *Binary relevance* ignores label correlations in the linear models; classifier chains would exploit them, as the transformer's shared encoder does.
- *The transformer was run once, with fixed hyper-parameters* (two epochs, learning rate $3 \cdot 10^{-5}$, 192 tokens). Larger encoders such as RoBERTa, longer sequences, or a class-weighted loss for the rare labels are natural next steps.
- *The two model families were evaluated in separate runs*, on different machines and library versions, and merged (`scripts/merge_runs.py`). The script checks that both runs used the same corpus, seed and fold strategy; the linear models' per-fold scores were reproduced identically on both machines, which confirms that the folds coincide.
- *Annotation noise* bounds the attainable score, as the error analysis shows.
- The bias audit uses a short, English-only list of identity terms, and some subgroups are small (31 comments mention *transgender*).

9 Conclusion

On the Jigsaw corpus a fine-tuned DistilBERT reaches a mean column-wise ROC-AUC of 0.9898, and a well-configured linear model over word and character TF-IDF features 0.9841 at a tenth of the cost; both are far above a rule-based baseline. The transformer's advantage is concentrated where context matters: the rare labels, profanity that is quoted rather than aimed at someone, and the label hierarchy, which it learns without being told.

The headline number, however, is not where the difficulty lies. The rare labels remain hard, many of the remaining errors are labelling mistakes or sarcasm that neither model understands, and both models are markedly biased against comments that mention identity groups – the transformer less than the linear model, but still flagging a clean comment about gay people almost seven times as often as an average one. Any deployment would have to address that bias before the accuracy figures become relevant.

A Reproducibility

All results were produced with random seed 42 and iterative stratification into 5 folds. Table 8 lists the software environment of the linear-model run. The DistilBERT run used Python 3.10.11 with scikit-learn 1.7.2 on an NVIDIA RTX 4050 Laptop GPU. To reproduce:

```
pip install -r requirements-dev.txt
python scripts/download_data.py
python scripts/run_experiment.py --folds 5 --tag experiment
# on a machine with a CUDA GPU and requirements-dl.txt installed:
python scripts/run_experiment.py --models distilbert --folds 5 --tag dl
python scripts/merge_runs.py --base experiment --extra dl
python scripts/make_report_assets.py && python scripts/make_report_figures.py
```

Table 8: Software and hardware of the linear-model run.

Component	Version
Python	3.14.0
Platform	Windows-11-10.0.26300-SP0
Processor	Intel64 Family 6 Model 141 Stepping 1, GenuineIntel
joblib	1.6.0
numpy	2.5.4
pandas	3.0.6
scipy	1.18.1
sklearn	1.9.1
Random seed	42

B Division of work

The work was split into four areas that map onto disjoint parts of the repository, so that the Git history attributes each part to its author.

Member	Area	Main files
(name)	Data and persistence	`src/dataio/`, `src/repository/`, `scripts/download_data.py`, notebook 01
(name)	Features and classical models	`src/preprocessing/`, `src/models/linear.py`, `src/models/dummy.py`, `src/models/registry.py`
(name)	Deep learning	`src/models/transformer.py`, the GPU runs
(name)	Evaluation, analysis and report	`src/evaluation/`, `scripts/`, notebooks 02 and 03, `report/`

Each member wrote the tests for their own area.

References

[1] S. Wang and C. D. Manning. Baselines and bigrams: simple, good sentiment and topic classification. In *Proceedings of ACL*, 2012.

[2] V. Sanh, L. Debut, J. Chaumond and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019.

[3] K. Sechidis, G. Tsoumakas and I. Vlahavas. On the stratification of multi-label data. In *ECML PKDD*, 2011.

[4] Jigsaw and Google. Toxic Comment Classification Challenge. `https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge`, 2018.